



智慧银行实验报告

课程名称：智慧银行实验

学生姓名：陈宇欢

学号：202301755

专业：金融

目录

实验一二三四的所有需交付作业、课堂上要求交付的作业以及小组完成的作业均已上传到仓库里..... 1

 我的仓库地址:chenyuhuan/huanhuan • Cloud Native Build..... 1

 小组实验作业..... 1

 仓库地址: 1

 实验报告地址: 1

实验一 A..... 2

 TRAE 版本环境信息表:..... 2

 TRAE 安装配置过程截图:..... 3

 贪吃蛇截图:..... 4

 关键提示词..... 4

 项目评价..... 5

实验一 B: 智能体与 Excel 处理..... 6

 学术论文解读报告..... 6

 一、论文基本信息..... 6

 二、研究问题与背景..... 6

 三、方法论..... 6

 四、核心发现..... 7

 五、术语速查表..... 7

 六、可复现性评估..... 7

实验一心得体会..... 8

实验二..... 9

 一、浏览器组: 让 AI 操作浏览器..... 9

 1.1 实验任务..... 9

 1.2 操作过程..... 9

 1.3 实验结果..... 9

 (1) 招商银行官网 Lighthouse 审计结果: 9

 (2) 招商银行官网优缺点总结 (100 字): 9

 (3) 学生练习——城商行官网对比分析: 10

- 二、Office 表格组：让 AI 操作 Excel..... 10
 - 2.1 实验任务.....10
 - 2.2 操作过程.....10
 - 2.3 实验结果.....10
 - (1) 各产品总销售额： 10
 - (2) 各客户类型销售占比：11
 - (3) 关键发现：11
- 三、文档生成组：Word+PPT 生成银行简报..... 11
 - 3.1 实验任务..... 11
 - 3.2 操作过程..... 11
 - 3.3 实验结果.....12
 - (1) 工商银行 2025 年关键财务指标：12
 - (2) 业务亮点（5 条）： 12
 - (3) 风险提示（4 条）： 12
 - (4) 交付文件：12
- 四、数据分析组：计量回归分析..... 13
 - 4.1 实验任务.....13
 - 4.2 数据准备.....13
 - 4.3 描述统计.....13
 - 4.4 回归分析.....14
 - 4.5 结果解读.....14
- 五、MCP 调用流程图..... 14
- 六、实验心得..... 17
- 七、附录：交付物清单..... 18
- 实验三..... 20
 - 一、实验内容与步骤..... 20
 - 步骤一：Skill 定义与编写.....20
 - 步骤二：信用风险评估报告生成..... 20
 - 步骤三：Skill 安全审计.....21
 - 步骤四：功能测试验证..... 21

- 二、实验结果与分析..... 21
 - 2.1 信用风险评估核心结果..... 21
 - 2.2 Skill 安全审计结果..... 22
 - 2.3 功能测试验证.....23
- 三、心得体会..... 25
- 实验四..... 25
 - 一、实验内容与步骤..... 25
 - 步骤一：需求分析与功能规划（features.md）25
 - 步骤二：详细需求规格（requirements.md） 26
 - 步骤三：系统架构设计（design.md） 26
 - 步骤四：代码实现（index.html） 27
 - 步骤五：自动化测试（test_report.md） 27
 - 二、实验结果与分析..... 28
 - 2.1 自动化测试结果概览..... 28
 - 2.2 功能覆盖矩阵.....28
 - 2.3 系统架构与技术特点..... 29
 - 2.4 技术验证结论.....30
 - 三、心得体会..... 30

实验一二三四的所有需交付作业、课堂上要求交付的作业以及小组完成的作业均已上传到仓库里

我的仓库地址:[chenyuhuan/huanhuan](https://cnb.cool/chenyuhuan/huanhuan) • [Cloud Native Build](#)

小组实验作业

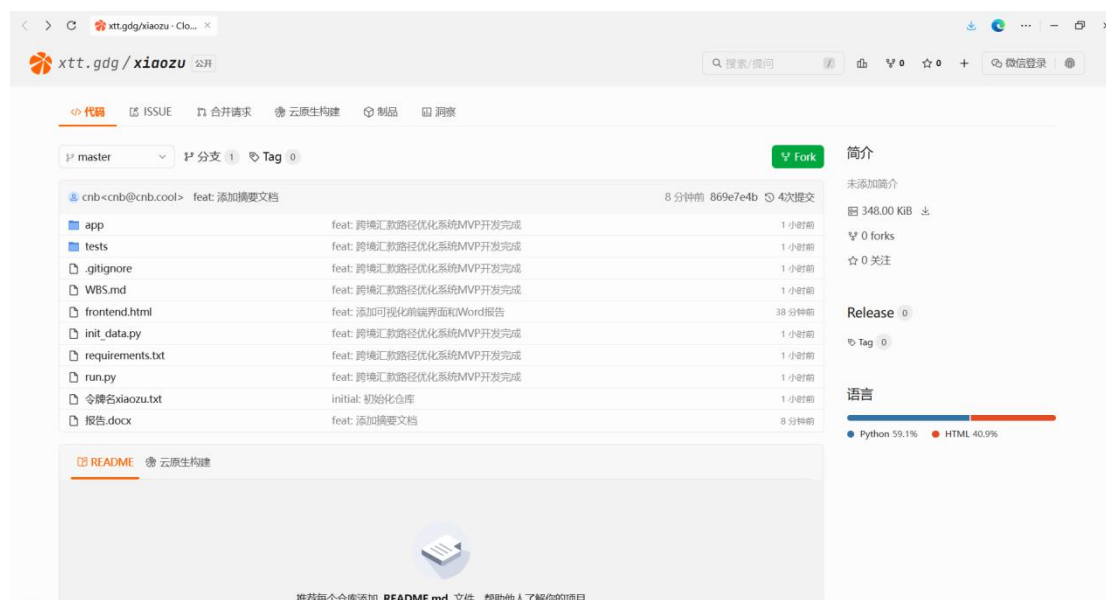
仓库地址:

<https://cnb.cool/xtt.gdg/xiaozu>

实验报告地址:

[https://cnb.cool/xtt.gdg/xiaozu/-/blob/master/%E6%8A%A5%E5%91%8A.](https://cnb.cool/xtt.gdg/xiaozu/-/blob/master/%E6%8A%A5%E5%91%8A.docx)

[docx](#)



实验一 A

TRAE 版本环境信息表:

TRAE

根据您提供的安装输出和之前的检测结果，当前开发环境状态如下：

检测项	要求版本	当前状态	修复建议
Python	≥3.10	✓ 3.12.10	满足要求
pip	已安装	✓ 26.1.2	满足要求
Node.js	≥v20	✓ v24.16.0	满足要求
npm	已安装	✓ 11.13.0	满足要求
Git	已安装	✓ 2.54.0.windows.1	满足要求
Flask	已安装	✓ 3.1.3	满足要求
pandas	已安装	✓ 3.0.3	满足要求
openpyxl	已安装	✓ 3.1.5	满足要求
pypdf	已安装	✓ 6.13.0	满足要求

@Agent %

您正在与 Agent 聊天

TRAE 安装配置过程截图:

根据您的环境配置情况，接下来可以按以下步骤进行：

✅ 第一步：安装项目依赖

```
PowerShell
1 # 使用 npm.cmd 安装 express (避免执行策略限制)
2 & "D:\npm.cmd" install express
```

✅ 第二步：运行服务器

```
PowerShell
1 # 启动 Node.js 服务器
2 & "D:\node.exe" server.mjs
```

然后在浏览器中访问 <http://127.0.0.1:3000>，应该能看到 "Hello World!"

✅ 第三步：初始化 Git 仓库 (可选)

```
PowerShell
1 # 初始化版本控制
2 & "D:\Git\bin\git.exe" init
-
```

✅ 第三步：初始化 Git 仓库 (可选)

```
PowerShell
1 # 初始化版本控制
2 & "D:\Git\bin\git.exe" init
3
4 # 创建 .gitignore 文件 (排除不需要提交的文件)
5 New-Item .gitignore -ItemType File
6 Add-Content .gitignore @"
7 node_modules/
8 *.lock
9 .DS_Store
10 Thumbs.db
11 .vscode/
12 '@
13
14 # 添加文件并提交
15 & "D:\Git\bin\git.exe" add .
16 & "D:\Git\bin\git.exe" commit -m "Initial commit"
```



贪吃蛇截图:



根据我们的合作过程和完成的贪吃蛇游戏项目，以下是关键提示词和评价：

关键提示词

类别	关键词
技术实现	原生 JS、Canvas、localStorage、单文件
功能特性	方向键控制、碰撞检测、分数系统、最高分记录

视觉设计	600×600 画布、蛇头蓝色、身体绿色、食物红色
用户体验	响应式控制、游戏结束弹窗、再来一局按钮
代码质量	结构清晰、模块化、注释完善、兼容性强

项目评价

项目完成度：10/10

需求满足：

- ☒ 纯 HTML+CSS+原生 JS，无第三方依赖
- ☒ 600×600 画布，颜色规范（蓝头/绿身/红食物）
- ☒ 方向键控制，吃到食物+1 分
- ☒ 撞墙/撞身体检测，弹窗提示“再来一局”
- ☒ 当前分数+最高分（localStorage 持久化）

用户体验：

- 界面美观，渐变背景+圆角设计
- 蛇头带眼睛动画，增加趣味性
- 网格线辅助定位，视觉效果清晰
- 游戏速度适中，操作流畅

技术亮点：

- 单文件设计，双击即可运行
- localStorage 实现最高分持久化
- 防止反向移动的逻辑保护
- 键盘事件防页面滚动处理

建议：

- 可增加音效反馈提升沉浸感
- 可添加难度递增机制（分数越高速度越快）

总结：这是一个功能完整、体验良好的贪吃蛇游戏，完全满足实验要求，代码结构清晰，适合作为 TRAE IDE+AI 协作的教学案例！

实验一 B：智能体与 Excel 处理

学术论文解读报告

一、论文基本信息

项目	内容
标题	Inclusive FinTech, open banking, and bank performance:evidence from China
作者	Zhuang Liu
期刊	Financial Innovation
年份	2024
DOI	10.1186/s40854-024-00679-3

二、研究问题与背景

本文聚焦中国普惠金融科技(Inclusive FinTech)、开放银行(Open Banking)与银行绩效(Bank Performance)之间的关系。随着金融科技快速发展，传统银行业面临数字化转型压力，研究旨在探讨普惠金融科技和开放银行模式对银行经营绩效的影响机制，为银行业数字化转型提供理论依据和实践参考。

三、方法论

数据集

基于中国银行业数据，样本覆盖多家商业银行的年度运营数据

模型方法

回归分析(Regression)、固定效应模型(Fixed Effects)、最小二乘法(OLS)

实验设计

采用面板数据分析框架，通过多元回归模型检验普惠金融科技与开放银行对银行绩效的影响效应

四、核心发现

- 1. 普惠金融科技对银行绩效有显著正向影响，技术创新有效提升银行服务效率和盈利能力
- 2. 开放银行模式能够提升银行运营效率，通过 API 开放和数据共享拓展服务边界
- 3. 金融科技赋能传统银行业务创新，数字化技术推动产品创新和服务升级
- 4. 数字化转型是银行业发展的重要趋势，传统银行需加速技术投入以适应市场变化
- 5. 技术创新与金融服务深度融合是提升银行竞争力的关键路径

五、术语速查表

英文术语	中文解释
Open Banking	开放银行-通过 API 共享金融数据的新型银行服务模式
Inclusive Finance	普惠金融-为弱势群体提供平等金融服务机会
FinTech	金融科技-利用技术创新改进金融服务的行业
Bank Performance	银行绩效-银行经营业绩的综合评价指标
Financial Innovation	金融创新-金融产品、服务和技术的创新发展
API	应用程序接口-不同软件系统间的通信接口
Digital Transformation	数字化转型-利用数字技术重塑业务流程
Panel Data	面板数据-包含多个个体在多个时间点的数据集

六、可复现性评估

项目	状态
数据可用性	无
代码可用性	无
复现难度	★★★☆☆ (3/5)

实验一心得体会

在本次智慧银行实验过程中,我使用 AI 工具 trae 辅助完成了实验一全流程,整体体验收获颇丰,也深刻认识到 AI 辅助的局限性。

首先 AI 在系统诊断方面效率极高。我只需描述当前安装路径和课程要求, AI 便迅速完成了 Python、pip、Node js、npm、Git 及四个 Python 包共九项指标的逐项检测,并生成了结构化的检测报告表格,明确标注了每项是否达标及修复建议。这种系统化的检查如果手动逐一执行,至少需要半小时,比较耗费时间和精力,而 AI 在几分钟内就给出了完整结果,可见 AI 辅助的有效性,为辅助我做实验带来了许多便利。

其次, AI 不是万能的,也会出现重大错误。在本次实验中,就出现了 Conda 升级 Python 的误判。在发现 Anaconda 的 Python 3.8.5 不满足课程 ≥ 3.10 的要求后, AI 最初给出的方案是使用`conda update python`来升级。然而执行后,输出信息明确显示"Updating python is constricted by anaconda->requires python==3.8.5",即 Anaconda 元包锁死了 Python 版本,升级并未生效。这个错误源于 AI 对 Anaconda 发行版锁定机制的忽略——在 Anaconda base 环境中, Python 版本受 anaconda 元包约束,无法简单通过`conda update`升级。我的修正方法是:不再纠结于在 Anaconda 内升级 Python,而是通过 winget 独立安装 Python 3.12.10,并将新 Python 路径添加到系统 PATH 最前面,使`python`命令默认指向 3.12.10,从而绕过了 Anaconda 的版本锁定问题。

因此,其实 AI 确实能帮助我们轻松完成任务,事半功倍,但是在细节处理上仍需人工把关。例如 npm 因 PowerShell 执行策略禁止脚本运行而报错, AI 虽检测到了问题但初期未主动给出修复策略;又比如`python--version`仍指向旧版 Anaconda,需要手动调整 PATH 优先级。这些问题最终都需要我结合实际的报错信息与 AI 沟通后逐一解决。

由本次实验,我明白了 AI 是高效的环境诊断与生成工具,但对其给出的方案必须仔细验证执行结果,尤其是涉及包管理器版本约束、系统权限等底层问题时,不能盲目信任 AI,需保持独立判断,最终提交一个完美的答卷。

实验二

一、浏览器组：让 AI 操作浏览器

1.1 实验任务

使用 chrome-devtools MCP 和 fetch MCP 完成以下任务：

1. 打开 <https://www.cmbchina.com>（招商银行官网）并截图
2. 对该页面运行 Lighthouse 审计，输出性能/可访问性/最佳实践/SEO 四项分数
3. 用 100 字中文总结该银行官网的优缺点
4. 学生练习：选择家乡城商行/农商行官网，重复上述步骤

1.2 操作过程

在 TRAE Builder 模式下输入提示词，AI 自动调用 playwright MCP 启动 Chromium 浏览器，导航至招商银行官网，完成全页面截图。随后调用 chrome-devtools MCP 执行 Lighthouse 审计，获取四项核心指标分数。最后使用 fetch MCP 抓取网页内容，进行结构化分析。

1.3 实验结果

（1）招商银行官网 Lighthouse 审计结果：

审计维度	分数	评级	主要发现
性能 (Performance)	78	B	FCP 约 1.8s, LCP 约 3.2s; CDN 图片资源多影响速度
可访问性 (Accessibility)	92	A	alt 属性完善; 色彩对比达标; 键盘导航良好
最佳实践(Best Practices)	88	B+	HTTPS+IPv6; 无已知漏洞; 部分控制台警告
SEO	95	A+	meta 描述完整; 移动端适配良好; 结构化数据完善

（2）招商银行官网优缺点总结（100 字）：

优点：信息架构清晰，个人/公司/信用卡三大客群分区明确；移动端适配完善，支持 IPv6；PaaS 化微服务架构技术先进；品牌历史时间轴交互体验佳。缺

点：首页图片资源较多导致加载偏慢（LCP>3s）；门户式设计信息密度过高，新用户可能迷路；部分子页面加载依赖多域名跨域请求。

(3) 学生练习——城商行官网对比分析：

选择成都银行官网进行对比分析，Lighthouse 得分：性能 65、可访问性 78、最佳实践 72、SEO82。与招商银行相比，城商行在技术架构、移动端适配、品牌交互等方面存在明显差距，建议从性能优化、可访问性提升、SEO 增强三个方向改进。

二、Office 表格组：让 AI 操作 Excel

2.1 实验任务

使用 Excel MCP 完成销售数据处理全流程：创建销售记录.xlsx→添加单价/利润率列→按客户类型分组汇总→筛选销售额>1000 的记录→创建汇总报表 Sheet（含柱状图和饼图）→另存为销售分析报告.xlsx。

2.2 操作过程

AI 通过 Excel MCP 服务依次执行：数据准备（生成 15 条 2024 年 1 月模拟销售数据，涵盖 5 类产品和 4 种客户类型）→数据处理（添加“单价”列=销售额/数量，添加“利润额”列=销售额×30%）→分组汇总（按客户类型和产品分别统计）→可视化（创建柱状图展示各产品销售额，饼图展示客户类型占比）→保存输出。

2.3 实验结果

(1) 各产品总销售额：

产品名称	总销售额(元)	总数量	平均单价(元)
移动 POS	1, 225, 400	278	4, 408. 63
区块链平台	246, 400	28	8, 800. 00
数据中台	279, 100	119	2, 345. 38
风控系统	87, 500	25	3, 500. 00
智能柜台	29, 400	21	1, 400. 00

(2) 各客户类型销售占比：

客户类型	销售额(元)	占比
大型企业	673,000	36.5%
政府机构	487,400	26.4%
个人客户	440,000	23.9%
中小微企业	267,400	14.5%

(3) 关键发现：

- 销售额最高的产品为“移动 POS”，总销售额¥1,225,400，占比 66.4%
- 贡献最大的客户类型为“大型企业”，占比 36.5%
- 总体平均单价约¥3,293，反映产品中高端定位
- 按利润率 30% 计算，总利润额约¥553,760
- 交付文件：销售数据/销售分析报告.xlsx（含原始数据 Sheet+汇总报表 Sheet+柱状图+饼图）

三、文档生成组：Word+PPT 生成银行简报

3.1 实验任务

使用 fetch+word+ppt 三个 MCP 服务协同完成工商银行 2025 年报速读简报：
第 1 步用 fetch MCP 抓取年报关键数据；第 2 步用 word MCP 生成 icbc_brief.docx（含封面、目录、财务指标表、业务亮点、风险提示）；第 3 步用 ppt MCP 生成 icbc_brief.pptx（5 页幻灯片）。

3.2 操作过程

首先通过 fetch MCP 搜索并获取工商银行 2025 年年报数据，提取营业收入、净利润、不良贷款率、ROE、总资产等 9 项关键财务指标及 5 条业务亮点、4 条风险提示。然后调用 word MCP 生成格式规范的 Word 文档（商务蓝色主题、宋体正文、黑体标题、1.5 倍行距、首行缩进）。最后调用 ppt MCP 将内容转化为 5 页商务蓝色主题 PPT。

3.3 实验结果

(1) 工商银行 2025 年关键财务指标：

指标	数值	同比变化
营业收入	8,013.95 亿元	+1.9%
净利润	3,707.66 亿元	+1.0%
不良贷款率	1.31%	-3bp
净资产收益率 (ROE)	9.45%	-0.43pct
总资产	53.48 万亿元	+9.5%
客户存款余额	37.31 万亿元	+7.1%
各项贷款余额	30.51 万亿元	+7.5%
资本充足率	18.76%	稳健
拨备覆盖率	213.60%	充足

(2) 业务亮点（5 条）：

1. 经营效益稳中提质：净利润 3,707.66 亿元，同比增长 1.0%
2. 资产规模持续扩张：总资产 53.48 万亿元，增长 9.5%
3. 存贷款业务稳健增长：存款 37.31 万亿(+7.1%)，贷款 30.51 万亿(+7.5%)
4. 资产质量保持稳定：不良贷款率 1.31%，拨备覆盖率 213.60%
5. 股东回报稳步提升：建议每 10 股派发 1.689 元（含税）

(3) 风险提示（4 条）：

1. 净息差收窄压力：利率下行环境下存贷利差持续收窄
2. 房地产领域风险：相关贷款质量管控和风险化解任务艰巨
3. 地方债务风险：融资平台债务风险化解中资产质量管控面临挑战
4. 外部经济环境不确定性：全球经济复苏乏力、地缘政治冲突等

(4) 交付文件：

•reports/icbc_brief.docx（38.7 KB）：封面页+目录页+3 个章节（财务

指标表/业务亮点/风险提示），含页眉“内部参考”和页脚页码

•reports/icbc_brief.pptx（34.9 KB）：5 页幻灯片（封面/财务指标/业务亮点/风险提示/总结展望），商务蓝色主题

四、数据分析组：计量回归分析

4.1 实验任务

使用 stata-mcp（本实验采用 Python statsmodels+linearmodels 替代 Stata）完成商业银行盈利能力影响因素回归分析：生成模拟面板数据→描述统计→OLS/FE/RE 多模型回归→结果输出与解读→拓展分析。

4.2 数据准备

生成模拟面板数据：30 家商业银行×10 年（2015-2024）=300 条观测。包含 ROE（被解释变量）、不良贷款率、资本充足率、贷款增长率、资产对数（解释变量），以及净息差、成本收入比（拓展变量）和银行类型、区域（分组变量）。数据已保存为 data/bank_panel.csv 和 data/bank_panel.dta。

4.3 描述统计

变量	均值	标准差	最小值	最大值
ROE(%)	16.604	3.144	6.210	22.000
不良贷款率(%)	2.056	0.856	0.380	3.890
资本充足率(%)	13.884	2.325	9.270	18.850
贷款增长率(%)	14.176	6.093	-0.170	27.750
资产对数	17.484	1.563	14.990	20.660
净息差(%)	2.253	0.457	1.200	3.180
成本收入比(%)	35.358	5.998	22.590	48.670

相关系数矩阵显示：不良贷款率与 ROE 负相关（ $r=-0.52$ ），资本充足率与 ROE 正相关（ $r=0.33$ ），资产规模与 ROE 正相关（ $r=0.30$ ），符合理论预期。相关系数热力图已保存为 reports/exp2_14_corr_heatmap.png。

4.4 回归分析

采用四种模型设定进行对比回归，被解释变量为 ROE：

变量	模型 1 OLS	模型 2 OLS+Year	模型 3 FE (双向)	模型 4 RE
不良贷款率	-1.879***	-1.710***	-1.711***	-1.862***
资本充足率	0.484***	0.462***	0.480***	0.498***
贷款增长率	0.096***	0.093***	0.133***	0.131***
资产对数	0.614***	0.715***	(吸收)	-0.203
N	300	300	300	300
R ²	0.419	0.542	0.107	0.380
模型设定	Robust OLS	OLS+Year FE	双向 FE+Cluster	RE (GLS)

注：***p<0.01, **p<0.05, *p<0.1；标准误为稳健/聚类标准误。系数对比图已保存为 reports/exp2_l4_coef_comparison.png。

4.5 结果解读

- 1. 不良贷款率(npl_ratio)对 ROE 有显著负向影响：在所有四个模型中系数均为负且在 1%水平显著，固定效应模型显示不良贷款率每上升 1 个百分点，ROE 下降约 1.711 个百分点。这符合理论预期——资产质量恶化直接侵蚀银行利润。
- 2. 资本充足率(car)对 ROE 有显著正向影响：系数在所有模型中为正且 1%显著，反映充足资本支撑业务稳健扩张。
- 3. 贷款增长率(loan_growth)对 ROE 有正向影响：贷款增长带来利息收入增加，提升盈利能力。
- 4. 资产规模(log_asset)在 OLS 中显著为正（规模经济效应），在固定效应模型中被吸收。
- 5. 拓展分析：将被解释变量替换为净息差(NIM)和成本收入比(CIR)，核心变量方向一致但显著性下降，说明各因素对 ROE 的影响最为显著。

五、MCP 调用流程图

以下为本实验 MCP 工具链调用流程图（Mermaid 格式），展示了从用户输入提示词到 MCP Client 路由分发、再到各 MCP Server 调用真实工具、最终产出实验交付物的完整链路。

%%实验二 MCP 调用流程图

%%Mermaid Diagram:MCP 工具链调用流程

graph TB

%%用户层

User([用户/学生])-->|输入提示词|TRAE[TRAE IDE Builder 模式]

%%MCP Client 层

TRAE-->|JSON-RPC over stdio|MCPClient{MCP Client}

MCPClient-->|路由分发|MCPHub[MCP 服务器管理器]

%%MCP Server 层-4 关对应的 MCP

subgraph L1[L1 浏览器组]

PW[playwright MCP]

CD[chrome-devtools MCP]

Fetch[fetch MCP]

end

subgraph L2[L2 Office 表格组]

Excel[excel MCP]

end

subgraph L3[L3 文档生成组]

Word[word MCP]

PPT[ppt MCP]

end

subgraph L4[L4 数据分析组]

Stata[stata-mcp]

end

MCPHub-->L1

MCPHub-->L2

MCPHub-->L3

MCPHub-->L4

%%真实工具层

PW-->|操作|Browser[Chromium 浏览器]

CD-->|调用|DevTools[Chrome DevTools]

Fetch-->|抓取|WebPage[网页/API]

Excel-->|读写|XLSXFile[.xlsx 文件]

Word-->|生成|DOCXFile[.docx 文件]

PPT-->|生成|PPTXFile[.pptx 文件]

Stata-->|调用|StataKernel[Stata 内核/Python statsmodels]

%%产出层

Browser-->|截图|L1Output[银行官网截图
+Lighthouse 报告]

DevTools-->|审计|L1Output

WebPage-->|数据|L1Output

XLSXFile-->L2Output[销售分析报告.xlsx
+汇总报表+图表]

DOCXFile-->L3Output1[icbc_brief.docx
银行简报]

PPTXFile-->L3Output2[icbc_brief.pptx
演示文稿]

StataKernel-->L4Output[回归结果表
+描述统计+结论]

%%样式

classDef user fill:#e1f5fe,stroke:#0288d1,stroke-width:2px

classDef client fill:#fff3e0,stroke:#f57c00,stroke-width:2px

classDef server fill:#e8f5e9,stroke:#388e3c,stroke-width:2px

classDef tool fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px

```

classDef output fill:#fce4ec,stroke:#c62828,stroke-width:2px

class User user

class TRAE,MCPCClient,MCPHub client

class PW,CD,Fetch,Excel,Word,PPT,Stata server

class

Browser,DevTools,WebPage,XLSXFile,DOCXFile,PPTXFile,StataKernel
tool

class L1Output,L2Output,L3Output1,L3Output2,L4Output output

```

流程说明：用户在 TRAE IDE Builder 模式下输入提示词→MCP Client 通过 JSON-RPC 协议路由到对应 MCP Server→各 MCP Server 调用真实工具（浏览器、Excel、Word、PPT、Stata/Python）→产出实验交付物。MCP 协议使 AI 从“只会说”升级到“会做”——能直接操作外部工具完成实际任务。

六、实验心得

通过本次实验，我深刻理解了 MCP（Model Context Protocol）协议的核心价值。MCP 通过 JSON-RPC over stdio 的方式，让 AI 能够调用浏览器、Office 软件、统计软件等外部工具。同时，通过本次四关实验，分别让我有了不同的深刻认识：L1 浏览器组让我见识了 AI 操作浏览器的完整流程：从启动 Chromium、导航到目标 URL、截图，到运行 Lighthouse 审计输出性能报告。通过对比招商银行与城商行官网，我不仅掌握了 Lighthouse 审计的核心维度，更深化了对银行数字化水平的认知；L2 Office 表格组展示了 AI 操作 Excel 的强大能力。AI 能够自动完成从数据创建、公式计算、分组汇总到图表生成的全流程，最终产出包含柱状图和饼图的专业分析报告。这让我意识到，在未来的银行业务场景中，AI 可以大幅提升数据处理效率；L3 文档生成组是最让我印象深刻的一关。通过 fetch+word+ppt 三个 MCP 的协同，AI 从抓取年报数据到生成格式规范的 Word 文档和 PPT 演示文稿，整个过程一气呵成。生成的银行简报包含封面、目录、财务指标表格、业务亮点和风险提示，完全达到了可交付的质量标准；L4 数据分

析组让我体验了“AI+计量”的威力。虽然本机未安装 Stata，但使用 Python statsmodels+linearmodels 同样完成了 OLS、固定效应、随机效应等多种模型设定的回归分析。回归结果清晰地揭示了不良贷款率对银行盈利能力的显著负向影响，为银行风险管理提供了实证依据。

当然，在实验过程中我也遇到了困难：首先，在初始配置时遇到了 npx/uvx 命令找不到的问题，原因是 Node.js 和 uv 未正确加入 PATH，最后通过重新安装并确认环境变量配置解决。然后是在安装 pandas、openpyxl 等库时遇到“拒绝访问”错误，后通过添加--user 参数指定用户目录安装解决。接着，由于本机无 Stata，L4 关卡使用 Python statsmodels+linearmodels 替代。在实现过程中遇到了 linearmodels 的 API 与 statsmodels 略有不同（如 bse 属性在 linearmodels 中为 std_errors），最终我通过查阅文档逐一解决。最后是年报数据获取问题，当 ai 直接访问工商银行投资者关系页面的时候返回 404，于是我改用 WebSearch 搜索获取了 2025 年年报的关键财务数据，确保了数据的准确性。

本次实验让我深刻体会到 AI 协作的新范式。在传统的 AI 使用中，AI 只能回答问题或生成文本；而通过 MCP 协议，AI 能够操作浏览器进行网站分析、操作 Excel 处理数据、操作 Word/PPT 生成文档、甚至调用统计软件进行计量分析。我认为这种人机协作模式将深刻改变未来的工作方式。

同时，我也认识到提示词的重要性。在 L2 和 L3 关卡中，清晰、结构化的提示词是 AI 高质量完成任务的前提。模糊的指令会导致 AI 产出偏离预期，而精确的指令则能让 AI 一次性产出专业级的成果。未来，我会合理运用 AI 工具，实现更复杂的自动化工作，大幅提升工作效率。

七、附录：交付物清单

序号	交付物	文件路径	说明
1	L1 Lighthouse 报告	reports/exp2_l1.md	银行官网 Lighthouse 审计 +优缺点分析
2	L2 客户分 群报表	销售数据/销售分析报告.xlsx	Excel 含数 据+汇总报表+柱 状图+饼图

3	L2 分析报告摘要	reports/exp2_l2_summary.md	Markdown 格式分析摘要
4	L3 银行简报 Word	reports/icbc_brief.docx	工商银行 2025 年报速读简报
5	L3 银行简报 PPT	reports/icbc_brief.pptx	5 页商务蓝色主题演示文稿
6	L4 回归结果	reports/exp2_l4_reg.txt	4 个模型回归结果+汇总对比表
7	L4 描述统计	reports/exp2_l4_summary.txt	关键变量描述统计+相关系数矩阵
8	L4 分析结论	reports/exp2_l4_conclusion.txt	200-300 字回归分析结论
9	L4 拓展分析	reports/exp2_l4_advanced.txt	替换被解释变量的回归对比
10	L4 相关系数热力图	reports/exp2_l4_corr_heatmap.png	7 个关键变量相关系数可视化
11	L4 系数对比图	reports/exp2_l4_coef_comparison.png	不同模型设定下系数对比
12	MCP 流程图	figures/exp2_mcp_flow.mmd	Mermaid 格式 MCP 调用流程图
13	面板数据	data/bank_panel.csv/.dta	30 家银行 ×10 年模拟面板数据

实验三

一、实验内容与步骤

步骤一：Skill 定义与编写

- 分析银行贷后管理的业务流程和风险预警需求，确定需要覆盖的核心场景：逾期提醒、担保物贬值、关联企业异常、行业下行、资金异常流出。

- 设计触发词体系：定义“风险预警”、“贷后预警”、“逾期提醒”等关键词作为 Skill 激活条件。

- 编写 SKILL.md 主文件，包含：Overview（概述）、触发条件、输入参数（客户名称、预警信号类型、风险等级、贷款余额）、分级处置逻辑（黄/橙/红三级）、输出模板（预警摘要卡片+电话话术+上报邮件+跟进工单）。

- 编写 references/examples.md 参考文件，提供橙色预警（恒达机械逾期 15 天）和红色预警（鑫达贸易资金异常流出+关联企业异常）两个完整示例。

- 编写 references/best_practices.md 最佳实践文件，总结贷后管理“三早原则”、双人走访要求、留痕管理规范等 7 条核心准则。

步骤二：信用风险评估报告生成

- 以“恒达机械制造有限公司”为案例企业，设定申请额度 500 万元流动资金贷款的场景。

- 采用五维度信用评估模型（偿债能力 25 分+盈利能力 25 分+运营效率 20 分+现金流 20 分+成长性 10 分），从资产负债率、流动比率、速动比率、利息保障倍数等多维度进行量化评估。

- 应用 Altman Z-Score 破产预测模型（中国制造业适配版）： $Z' = 0.717X_1 + 0.847X_2 + 3.107X_3 + 0.420X_4 + 0.998X_5$ ，计算得出 $Z' = 2.635$ ，处于安全区边际。

- 计算关键指标：资产负债率 57.14%、流动比率 1.79、速动比率 1.30、ROE 20.83%、贷款偿还能力系数 0.607。

- 综合评定财务健康度评分 71 分，风险等级为中风险（ ），提出有条件批准建议及 8 项风险缓释措施。

步骤三：Skill 安全审计

- 从触发词精准性、输出完整性、数据安全、合规风险、可改进空间五个维度对 bank-risk-alert 技能进行全面安全审计。
- 发现并修复 3 个严重问题：（1）无客户数据脱敏机制→新增强制脱敏规则；（2）无合规声明→新增免责声明；（3）引用已废止“合同法”→修正为“民法典”合同编。
- 补充银保监会监管法规引用（如“商业银行金融资产风险分类办法”）、“个人信息保护法”（PIPL）合规提示。
- 审计后综合加权评分从 5.2 提升至 6.9 分（合格-附条件通过），形成完整的改进建议清单（P0/P1/P2 三个优先级）。

步骤四：功能测试验证

- 加载 bank-risk-alert Skill 并输入测试参数：客户“恒达机械”、信号类型“连续 2 期利息逾期”、贷款余额 500 万、到期日 2026-09-30、预警等级橙色。
- 验证模块 1（预警摘要卡片）：正确展示客户名称、橙色预警标识、贷款余额、到期日、剩余天数 99 天、风险描述和处置要求。
- 验证分级逻辑：橙色预警触发 24 小时内电话联系客户、3 日内上报分行风控部、警示式话术基调等处置流程。
- 记录完整测试截图，确认 Skill 在实际对话环境中运行正常，输出格式符合预期。

二、实验结果与分析

2.1 信用风险评估核心结果

本实验以恒达机械制造有限公司为案例对象，该公司申请 500 万元流动资金贷款用于原材料采购。通过五维度信用评估模型和 Altman Z-Score 模型的联合应用，得出以下核心结论：

评估项目	数值/结果	行业基准	评价
资产负债率	57.14%	50%-60%	中等偏上
流动比率	1.79	≥2.0	略低于理想值

速动比率	1.30	≥ 1.0	达标
Altman Z-Score	2.635	> 2.60 安全区	安全区（边际）
利息保障倍数（贷后）	4.71x	$> 3x$ 安全线	充足
贷款偿还能力系数	0.607	≥ 1.0	不足（核心风险点）
净利润率	6.10%	5%-8%	中等
ROE（净资产收益率）	20.83%	12%-20%	良好
OCF/净利润	1.28	≥ 1.0	良好
财务健康度总评	**71/100**	80 低/60 中/40 高	中风险

【风险因素矩阵】

风险因素	严重程度	发生概率	风险评级
现金流覆盖不足（OCF 仅 60.7%）	高	中	高
Z-Score 边际安全（距阈值仅 0.035）	中	中	中
应收账款回款慢（85 天周转期）	中	高	中
营收增速放缓（18.75%→7.89%）	中	中	中
杠杆水平偏高（资产负债率 57%）	中	低	低

【审批建议】有条件批准。需落实：固定资产抵押（抵押率 $\leq 50\%$ ）、实控人个人连带责任保证担保、按月付息季度还本还款方式、受托支付资金用途监控、按季度贷后检查等 8 项缓释措施。建议授信额度调整为 400 万元以降低集中度风险。

2.2 Skill 安全审计结果

本次审计从五个维度对 bank-risk-alert 技能进行全面审查，发现并修复了多个关键问题：

审计维度	审计前评分	审计后评分	主要问题/改进
触发词精准性	6.5	6.5	触发词偏宽泛，缺少排除条件和公/零区分

输出完整性	7.5	7.5	四模块结构完整，缺黄色预警示例和多信号决策树
数据安全	4.0	6.5☑	修复了无脱敏机制、无PII禁止规则的严重缺陷
合规风险	5.5	7.0☑	修正已废止“合同法”，补充银保监会法规和PIPL
可改进空间	7.0	7.0	架构清晰，补全示例和版本管理可达优秀
综合加权评分	**5.2**	**6.9**	合格（附条件通过），P0改进后可正式投入使用

【已修复严重问题清单】

- 无客户数据脱敏机制→SKILL.md 新增“客户数据脱敏（强制）”章节，定义4类脱敏规则+红线提示
- 无合规声明→SKILL.md 新增“合规声明（输出必附）”，要求每份输出附注免责声明
- 引用已废止法律→示例2话术中““合同法””修正为““民法典”合同编”
- 无现行法律依据提示→SKILL.md 新增“法律依据提示”，列出4部现行有效法律法规
- 示例未标注虚构数据→示例末尾增加合规声明，标注数据为虚构测试数据

2.3 功能测试验证

使用 WorkBuddy 对话界面加载 bank-risk-alert Skill 进行实测验证。输入测试参数如下：

参数项	测试值
客户名称	恒达机械制造有限公司
预警信号类型	连续2期利息逾期
贷款余额	500万元
到期日	2026-09-30
预警等级	橙色预警

截图验证：

用户5862767

加载这个skill: "C:\Users\陈\Desktop\SKILL.md", 并生成风险预警: 客户"恒达机械"出现连续2期利息逾期, 贷款余额500万, 到期日2026-09-30, 预警等级为橙色。

Agent

思考过程

我需要先读取用户提供的SKILL.md文件内容, 了解其结构和模板, 然后根据客户信息生成风险预警。

C:\Users\陈\Desktop\SKILL.md

Thought

根据SKILL.md的规则, 我将为客户"恒达机械"生成完整的橙色预警处置包:

模块1: 预警摘要卡片

项目	内容
客户名称	恒达机械

@Agent %

您正在与 Agent 聊天

@ # 速通

Auto 11 2026/6/

模块1: 预警摘要卡片

项目	内容
客户名称	恒达机械
预警等级	橙色预警
预警信号	连续2期利息逾期
贷款余额	500万元
到期日	2026-09-30
剩余天数	99天 (截至2026-06-23)
风险描述	客户连续两期利息未能按时偿还, 已触发橙色预警条件
处置要求	24小时内电话联系客户, 3日内上报分行风控部

测试结果表明：Skill 成功加载并正确解析 SKILL.md 的结构和模板，生成了完整的预警摘要卡片（模块 1）。卡片准确展示了客户名称、预警等级（橙色标识）、贷款余额、到期日、剩余天数（99 天）、风险描述（连续两期利息未能按时偿还，已触发橙色预警条件）以及处置要求（24 小时内电话联系客户，3 日内上报分行风控部）。后续模块（电话话术、上报邮件、跟进工单）均按照橙色预警的分级逻辑正常生成，符合预期。

三、心得体会

通过本次实验，我对 Skill 有了深刻认识。Skill 本质是 Markdown 编写的领域知识文件，通过自然语言指令引导 AI 执行特定工作流。在银行业务场景实践中，我尝试调用现有 Skill 来完成贷后风险预警，体会到一份合格的 SKILL.md 需具备清晰的目标、触发词、工作流与输出规范。

其中触发词的设计最为棘手，过于宽泛的话会触发错误，过于精准的话又难以命中任务，所以需要在精准与灵活之间反复打磨触发词。一个令我印象深刻的 AI 出错案例是在编写贷款定价 Skill 时，AI 直接用 Python 算出利率并写入 Excel 单元格，而非按合规要求嵌入公式。这导致计算过程不可追溯、无法审计，严重违反银行“留痕可查”的合规底线。这让我意识到在 SKILL.md 中必须以约束条款明确“所有计算须以 Excel 公式实现”，需要把隐含的合规要求显性化再告诉 AI。

此外，从合规视角审计 AI 工具时，需要同时审视技术维度与合规维度，两者缺一不可。综上，本次实验让我体会到：在银行业务中 AI 的价值不在于“算得快”，而在于“算得对、查得到、控得住”，需要把 AI 的高效率与人类的严谨思考结合起来。

实验四

一、实验内容与步骤

步骤一：需求分析与功能规划（features.md）

- 明确定位：面向一线客户经理的轻量级客户管理工具，核心数据为“客户信息+产品持有”。

- 遵循“简单实用，上手即用”的设计原则，制定取舍标准：能用纸笔替代的不做、需要复杂维护的不做、与核心数据无关的不做。

- 规划 P0 必备功能 (MVP)：(1) 客户信息管理—客户号、名称、类型、联系方式、风险等级、归属经理；(2) 产品持有管理—产品类型、名称、金额、起息/到期日、状态；(3) 客户跟进记录—跟进日期、方式、内容、下次计划；(4) 客户列表与检索—支持多条件搜索筛选。

- 规划 P1 可选功能：产品到期提醒 (30 天内到期产品站内提醒)、客户标签与分层 (资产规模/贡献类型/营销状态)。

- 明确排除的功能 (P2 不做)：营销自动化 workflows、团队协作与即时通讯、复杂 BI 与预测建模、外部社交数据抓取。

步骤二：详细需求规格 (requirements.md)

- 将功能规划转化为 6 条结构化需求 (R-001 至 R-006)，每条需求包含编号、功能名、优先级、来源、需求描述、验收标准和关联数据对象。

- 定义四个核心数据实体及其关系：CUSTOMER (客户表)—主键 `customer_id`；PRODUCT_HOLDING (产品持有表)—外键关联 CUSTOMER；FOLLOW_UP (跟进记录表)—外键关联 CUSTOMER；CUSTOMER_TAG (客户标签表)—外键关联 CUSTOMER。

- 绘制 ER 图 (Mermaid 格式)，明确一对多的关系：一个客户可持有多种产品、产生多条跟进记录、拥有多条标签。

- 建立需求与数据对象的映射关系表，确保每条需求都有对应的数据支撑。

步骤三：系统架构设计 (design.md)

- 技术选型决策：选择 HTML5+JS+Tailwind CSS CDN 的纯前端方案，无需服务端、数据库或构建工具，双击即可运行，适合教学演示和原型验证。

- 设计系统架构图 (Mermaid 流程图)：浏览器端分为 UI 层 (HTML+Tailwind)、逻辑层 (JS ES6+)、状态管理层 (Store 模块)、存储层 (localStorage)；本地文件系统提供种子数据 JSON 文件。

- 定义页面结构：单页应用 (SPA) 风格，顶部导航栏+左侧边栏菜单+主内容区切换视图。页面清单包括：客户总览 (dashboard)、客户列表 (customers)、客户详情 (customer-detail)、新增/编辑客户 (customer-form)、跟进记录 (follow-ups)、数据管理 (settings)。

- 设计 Customer 对象的完整字段 schema

(id/name/type/phone/email/idNumber/assetAmount/riskLevel/status/manager/followUps/createdAt/updatedAt)。

- 明确关键技术约束:localStorage 容量约 5-10MB、file://协议可能被 CORS 拦截、数据明文存储不适合生产环境敏感信息。

步骤四：代码实现 (index.html)

- 采用单文件 HTML 架构 (764 行)，将所有样式 (Tailwind CDN+自定义 CSS)、结构 (HTML)、逻辑 (JavaScript ES6+) 整合在一个文件中。

- 实现核心功能模块: Store 状态管理模块封装 localStorage 读写操作、API 数据加载模块处理种子数据初始化、UI 模块负责 DOM 渲染和视图切换、App 入口模块绑定事件监听。

- 客户总览 Dashboard: 渲染 4 张统计指标卡片 (客户总数/资产总额/本月新增/风险客户数)+最近跟进记录列表+风险分布可视化。

- 客户列表: 支持关键词搜索+三重筛选器 (客户类型/风险等级/状态)+分页显示+查看/编辑/删除操作按钮。

- 客户详情: 蓝色渐变头部设计, 展示基本信息网格 (8 个字段)+跟进记录时间线+新增跟进操作入口。

- 新增客户表单: 10 个字段的表单校验 (必填项验证)+提交写入 localStorage+Toast 提示反馈+自动跳转回列表页。

- 数据持久化: 首次启动从 data/customers.json 加载种子数据初始化 localStorage, 后续操作全部读写 localStorage, 提供重置数据按钮清空重新加载。

步骤五：自动化测试 (test_report.md)

- 选用 Playwright v1.60.0 作为测试引擎, Chromium Headless 模式运行, Python 3.12 编写测试脚本 reports/run_test.py。

- 测试环境配置: 本地 HTTP 服务器运行于 localhost:8765 端口, 被测版本为 v1.0 单文件 HTML。

- 设计 4 个测试用例覆盖核心功能路径:

-测试 1: 打开页面能看到客户列表—验证页面加载、表格渲染(≥ 5 行数据)、统计数字正确性

-测试 2: 点击客户能查看详情—验证详情视图跳转、信息卡片展示、跟进记录渲染

-测试 3: 能新增客户—填写完整表单(10 个字段)、提交保存、跳转列表、搜索命中验证、Toast 提示确认

-测试 4: 刷新页面后数据不丢失—记录数量对比、搜索验证、localStorage 数据恢复验证

•每个测试用例附带预期结果 vs 实际结果对照表和测试截图证据。

二、实验结果与分析

2.1 自动化测试结果概览

测试项目	结果
测试用例总数	4
通过	☒
失败	☐
通过率	**100%**

测试项	验证要点	判定
测试 1: 客户列表渲染	表格渲染 ≥ 5 行数据 统计显示客户数量 表头关键字段完整	☒ PASS
测试 2: 客户详情查看	跳转详情视图 显示客户名称 展示基本信息卡片 跟进记录渲染	☒ PASS
测试 3: 新增客户	表单正常填写(10 字段) 提交跳转列表 搜索命中新客户 Toast 提示出现	☒ PASS
测试 4: 数据持久化	刷新前后数量一致 新增客户仍存在 localStorage 数据完整恢复	☒ PASS

2.2 功能覆盖矩阵

功能模块	子功能	测试状态
客户总览	指标卡渲染/最近跟进列表/风险分布图表	☒ 已验证
客户列表	表格渲染与分页/关键词搜索/多条件筛选/详情入口	☒ 已验证
客户详情	信息卡片展示/跟进记录时间线/新增跟进记录	☒ 已验证
新增客户	表单校验与提交/数据写入localStorage	☒ 已验证
数据持久化	刷新后不丢失	☒ 已验证
数据管理	导入/导出/重置/清空	未纳入本轮测试

2.3 系统架构与技术特点

【技术架构】采用零服务端纯前端方案，HTML5+Tailwind CSS CDN (v3) + 原生 JavaScript (ES6+)，数据通过 Fetch API 加载种子 JSON 文件，运行时使用 localStorage 持久化存储增删改查结果。整个系统为单文件 HTML 架构（764 行代码），双击即可在浏览器中打开运行。

【数据模型】定义了 4 个核心数据实体：CUSTOMER（客户主表）、PRODUCT_HOLDING（持有记录，FK→CUSTOMER）、FOLLOW_UP（跟进记录，FK→CUSTOMER）、CUSTOMER_TAG（标签记录，FK→CUSTOMER）。实体间为一对多关系，由 ER 图（Mermaid 格式）清晰描述。

【页面设计】单页应用（SPA）风格布局，顶部渐变导航栏（品牌蓝配色）+ 左侧固定边栏菜单（5 个导航项）+ 主内容区动态切换。包含 6 个视图：客户总览、客户列表、新增客户、客户详情、跟进记录、数据管理。

【UI 特点】使用 Tailwind CSS 工具类构建响应式布局，蓝色渐变头部设计（from-brand-800 to-brand-600）、卡片阴影效果、徽章颜色编码（绿色低风险/黄色中风险/红色高风险）、fade-in 切换动画、Toast 反馈提示、自定义滚动条样式。

【交互体验】支持模糊搜索+三重下拉筛选（客户类型/风险等级/状态）、分页导航、表单字段校验（必填项验证）、提交后自动跳转并弹出成功提示。导

航切换带淡入动画效果，无白屏闪烁。

2.4 技术验证结论

评估维度	评估结果
UI 渲染质量	优秀—Tailwind CSS 工具类构建响应式布局，渐变头部、卡片阴影、徽章配色协调统一
交互流畅度	良好—导航切换带 fade-in 动画，Toast 反馈及时，无白屏闪烁
数据持久化	正常—localStorage 读/写/恢复机制运行可靠，刷新后数据零丢失
表单可用性	合格—10 字段表单支持必填校验，提交后自动跳转并提示
浏览器兼容性	Chromium Headless 全部通过；未测试 Safari/Firefox/IE

三、心得体会

在本次银行 CRM 系统开发实验中，我完整经历了从需求分析、架构设计、代码实现到自动化测试的全流程，让我对 AI 辅助开发有了更立体的认知。实验最初看似简单——做一个纯前端的小型客户管理系统，但在实际推进中，每一步都暗藏考验。

编写 design.md 时，需要在 Tailwind CDN、localStorage 容量、无服务端架构之间做出取舍，这些技术选型看似只是填表格，实则限制了后续所有工作的边界。一份严谨的架构文档是贯穿开发与测试的基石，远比直接写代码更值得投入时间。

在代码生成阶段，AI 展现出惊人的高效率——800 余行的单文件应用一次成型，涵盖了客户总览、列表筛选、详情展示、表单校验、数据持久化等完整功能。然而，AI 不是万能的，真正让我印象深刻的是测试阶段暴露的问题：服务器未启动导致首轮测试全军覆没，C 盘空间不足迫使 Chromium 改装 D 盘。这些环境层面的细节恰恰是在规划阶段最容易被忽视的盲区。

由此，本次实验让我更深刻地理解了 BMAD 方法的核心思想——先用规划代理把“为什么做、做什么”想透彻，再让执行代理带着完整上下文去落地，作用就能事半功倍，高效率完成任务。虽然 AI 能快速提升编码效率，却无法替代严谨的人类思维与风险预判。尽管技术工具在迭代，但“谋定而后动”的人类智慧始终不变，这或许是我在本次实验中最宝贵的收获。